同济大学

数据结构与算法设计 2
期中大作业报告

分治、动态规划与贪心算法

第 k 小元素 · 卡牌均分 · 最小生成树

学　号 2451951

姓　名 于沛可

专　业 数据科学与大数据技术

课　程 数据结构与算法设计 2

提交日期 2026 年 5 月

目录

1　编程语言与实验环境　3

2　第一题　分治算法：第 k 小元素　4

2.1　问题描述 4
2.2　三种算法 4
2.2.1　方法一：基于堆的选择 4
2.2.2　方法二：随机划分线性选择（QuickSelect） 4
2.2.3　方法三：中位数的中位数线性选择 4
2.3　代码实现（核心逻辑） 5
2.4　实验结果 6
2.5　分析与讨论 7

3　第二题　动态规划：宝可梦卡牌均分　9

3.1　问题描述 9
3.2　方法一：暴力枚举 9
3.3　方法二：动态规划 9
3.4　代码实现（核心逻辑） 10
3.5　实验结果 11
3.6　分析与讨论 11

4　第三题　贪心算法：供水管网最小生成树　13

4.1　问题建模 13
4.2　算法：Kruskal 13
4.3　代码实现 13
4.4　测试用例与结果 14
4.5　分析与讨论 15

5　其他说明　16

1　编程语言与实验环境

编程语言采用 C++（C++11 标准），编译器为 g++ 10.3.0（TDM-GCC 64-bit），运行平台为 Windows 10（x86-64）。时间测量使用 `std::chrono::high_resolution_clock`，单位为微秒（μs）。功能验证采用 `-O2` 编译；性能基准测试采用 `-O0`，以避免编译器消除死代码对结果的干扰。

2　第一题　分治算法：第 k 小元素

2.1　问题描述

给定无序整数序列 $A[1..n]$ 及整数 k（$1 \le k \le n$），求第 k 小元素，即排序后位于第 k 位的值。

2.2　三种算法

2.2.1　方法一：基于堆的选择

维护容量为 k 的大根堆 H：将前 k 个元素入堆后，对每个后续元素 a，若 $a < \mathrm{top}(H)$，则弹出堆顶并压入 a；遍历结束时 $\mathrm{top}(H)$ 即为答案。

时间复杂度　建堆 $O(k)$；后续 $n-k$ 次操作各需 $O(\log k)$，总计

$$T_1(n,k) = O(k + (n-k)\log k)\,.$$

当 $k \ll n$ 时，$\log k$ 较小，整体接近线性；当 $k = n/2$ 时，$\log k = \log(n/2)$，复杂度退化为 $O(n \log n)$。

2.2.2　方法二：随机划分线性选择（QuickSelect）

借鉴快速排序的分区思想，每次随机选取 pivot 并以此将 $A[l..r]$ 划分为三部分，根据 pivot 排名与 k 的关系仅对相关一侧递归。

时间复杂度　期望时间 $O(n)$。若用随机 pivot，则每步期望至少减少 $1/4$ 的元素，递推可得 $\mathbf{E}[T_2(n)] = O(n)$。最坏情形（每次选到极端元素）为 $O(n^2)$，但对均匀随机 pivot，其概率随 n 指数衰减，工程实践中几乎不出现。

2.2.3　方法三：中位数的中位数线性选择

将 $A[l..r]$ 每 5 个分组，各组内排序取中位数，再递归求这些中位数的中位数作为 pivot，从而保证每次划分后两侧规模各不超过 $\lceil 7n/10 \rceil + 6$。

时间复杂度　设 $T_3(n)$ 为最坏情形代价，则：

$$T_3(n) \le T_3(n/5) + T_3(7n/10 + 6) + O(n).$$

归纳可证 $T_3(n) = O(n)$，且对任意输入均成立，即这是一个**最坏情形** $O(n)$ 的算法。

2.3　代码实现（核心逻辑）

Listing 1: 方法一：基于大根堆的选择

```cpp
int heapSelect(vector<int> arr, int k) {
    priority_queue<int> pq;
    for (int i = 0; i < k; i++) pq.push(arr[i]);
    for (int i = k; i < (int)arr.size(); i++) {
        if (arr[i] < pq.top()) {
            pq.pop();
            pq.push(arr[i]);
        }
    }
    return pq.top();
}
```

Listing 2: 方法二：随机划分选择（QuickSelect）

```cpp
int randSelect(vector<int>& a, int l, int r, int k) {
    if (l == r) return a[l];
    int p = l + rand() % (r - l + 1);
    swap(a[p], a[r]);
    int pivot = a[r], i = l - 1;
    for (int j = l; j < r; j++)
        if (a[j] <= pivot) swap(a[++i], a[j]);
    swap(a[++i], a[r]);
    int cnt = i - l + 1;
    if (k == cnt) return a[i];
    return k < cnt ? randSelect(a, l, i-1, k)
                   : randSelect(a, i+1, r, k-cnt);
}
```

Listing 3: 方法三：中位数的中位数线性选择（递归核心）

```cpp
int medianSelect(vector<int>& a, int l, int r, int k) {
    if (r - l + 1 <= 5) {
        sort(a.begin()+l, a.begin()+r+1);
        return a[l+k-1];
    }
    vector<int> meds;
    for (int i = l; i <= r; i += 5) {
```

```
        int j = min(i+4, r);
        sort(a.begin()+i, a.begin()+j+1);
        meds.push_back(a[(i+j)/2]);
    }
    int mm = meds.size();
    int pivot = medianSelect(meds, 0, mm-1, (mm+1)/2);
    // 以 pivot 为基准划分，再对相应侧递归
    int pivotPos = l;
    for (int i = l; i <= r; i++)
        if (a[i] == pivot) { pivotPos = i; break; }
    swap(a[pivotPos], a[r]);
    int p = l-1;
    for (int j = l; j < r; j++)
        if (a[j] <= pivot) swap(a[++p], a[j]);
    swap(a[++p], a[r]);
    int rank = p - l + 1;
    if (k == rank) return a[p];
    return k < rank ? medianSelect(a, l, p-1, k)
                    : medianSelect(a, p+1, r, k-rank);
}
```

2.4　实验结果

功能验证　对测试数组 $\{3,1,4,1,5,9,2,6,5,3,5\}$，三种方法对所有 $k \in [1,11]$ 均给出正确结果，与排序后直接索引一致。

性能测试　以下两组实验分别考察运行时间随 n（固定 $k = n/2$）和随 k（固定 $n = 200{,}000$）的变化。每组实验生成同一随机数组，分别传副本给三种算法。

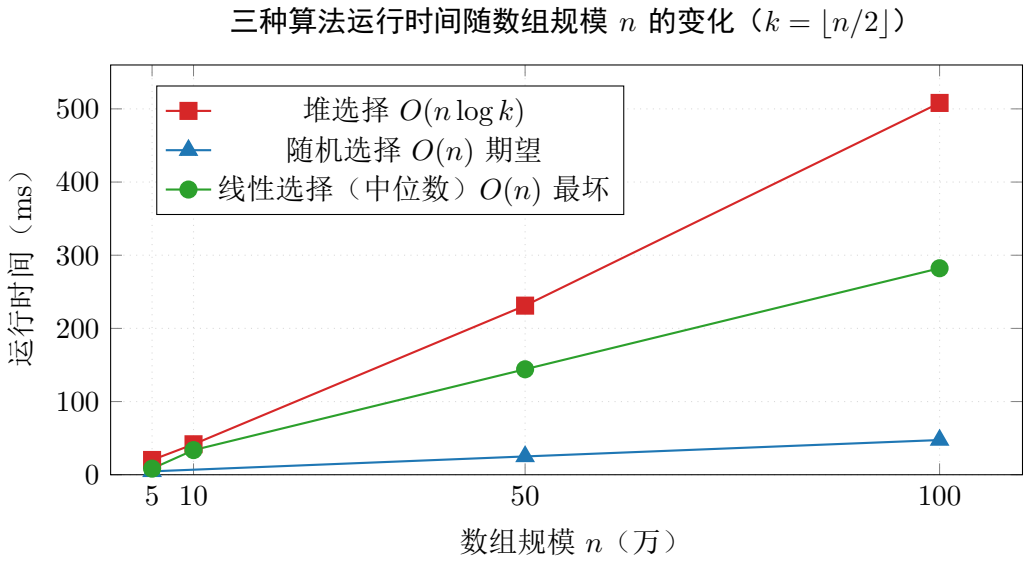


图 1: 运行时间随 n 的变化（$k = \lfloor n/2 \rfloor$，`-O0` 编译，单位 ms）

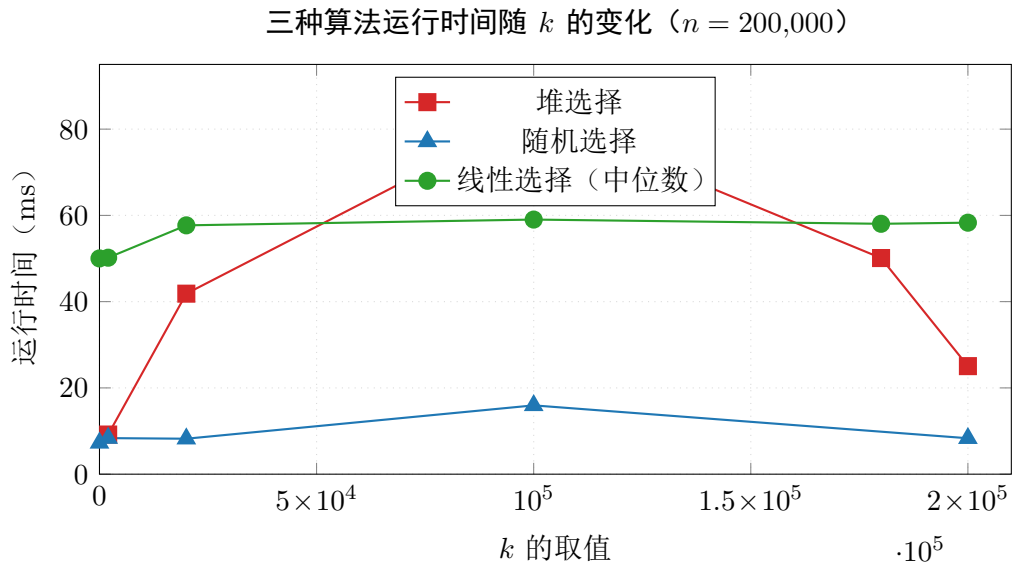


图 2: 运行时间随 k 的变化（$n = 200{,}000$，`-O0` 编译，单位 ms）

2.5　分析与讨论

三种方法的实际对比　图 1 中，$n = 100$ 万时随机选择（47 ms）约为中位数算法（282 ms）的 1/6，尽管二者理论复杂度同为 $O(n)$。常数因子的差异源于中位数算法需额外执行约 $n/5$ 规模的递归以及多轮分组排序，使每元素的平均操作数远高于 QuickSelect。因此，**最坏情形的渐近保证与平均实际性能之间存在显著权衡**：中位数算法适用于输入可能被对抗性构造的场景，而 QuickSelect 在随机数据下更具工程竞争力。

堆选择关于 k 的单峰特性　图 2 中，堆选择的曲线在 $k = n/2$ 附近达到峰值，两端（$k \ll n$ 或 $k \approx n$）均显著更快。这与复杂度公式 $O(k + (n-k)\log k)$ 完全一致：当 k 极小时，$\log k \approx 0$，堆操作代价趋近于零；当 $k \to n$ 时，$(n-k) \to 0$，同样减少操作量。

线性选择对 k 的不敏感性　线性选择在不同 k 值下的运行时间几乎恒定在 58 ms 左右，说明其常数因子主要由分组、提取中位数等与 k 无关的步骤主导。这一特性在 $k = 1$（求最小值）时最为显眼：此时堆选择只需 $O(n)$ 的单次扫描（$\log 1 = 0$），而中位数算法仍执行完整的 $O(n)$ 递归流程，代价相差约 7 倍。

3　第二题　动态规划：宝可梦卡牌均分

3.1　问题描述

给定 n 张卡牌，第 i 张价值为正整数 v_i，总价值 $S = \sum_{i=1}^{n} v_i$。将所有卡牌划分为两个不相交的子集，使两者价值之差最小。输出两堆各自的价值及所含卡牌编号（1-indexed）。

3.2　方法一：暴力枚举

算法描述　枚举 $\{0,1\}^n$ 中所有二进制掩码 m，第 i 位为 1 表示卡牌 i 归入第一堆。计算 $s = \sum_{i:\, m_i=1} v_i$，差值为 $|S - 2s|$，取差值最小的 m。

时间复杂度　$O(2^n \cdot n)$。当 $n \geq 30$ 时约需 10^{10} 次操作，实际不可行。

3.3　方法二：动态规划

问题转化　最小差值等价于在所有价值子集的和中，找到最接近 $\lfloor S/2 \rfloor$ 的可达和 s^*，则最优差值为 $S - 2s^*$。这是 0-1 背包的可行性变体。

状态定义　令 $dp[i][j] = \text{true}$ 当且仅当从前 i 张卡牌中，存在某个子集的价值恰为 j。

递推关系

$$dp[i][j] = dp[i-1][j] \ \vee \ \big(j \geq v_i \ \wedge \ dp[i-1][j - v_i]\big), \quad i \in [1, n],\ j \in [0, S]. \tag{1}$$

边界条件：$dp[0][0] = \text{true}$，其余为 false。

求解最优分配　填表后，令 $s^* = \max\{j \leq \lfloor S/2 \rfloor : dp[n][j] = \text{true}\}$，以逆序回溯（若 $dp[i-1][\cdot]$ 不能达到当前和，则第 i 张必在此堆中）还原具体分配。

时间与空间复杂度　$O(n \cdot S)$（时间）；$O(n \cdot S)$（存储二维表以供回溯）。若仅需最优值，可将二维数组压缩为一维（滚动更新），空间降至 $O(S)$。

DP 表格可视化　图 3 展示了样例（$n = 8$，$\mathbf{v} = (2, 1, 3, 1, 5, 2, 3, 4)$，$S = 21$，目标 $\lfloor S/2 \rfloor = 10$）中各行 $dp[i][j]$（$j \in [0, 10]$）的分布。蓝色格表示可达，灰色格表示不可达。关键观察：第 5 行（加入价值为 5 的卡牌后）首次使 $j = 10$ 变为可达，此后目标和始终保持可达，回溯在此行截止。

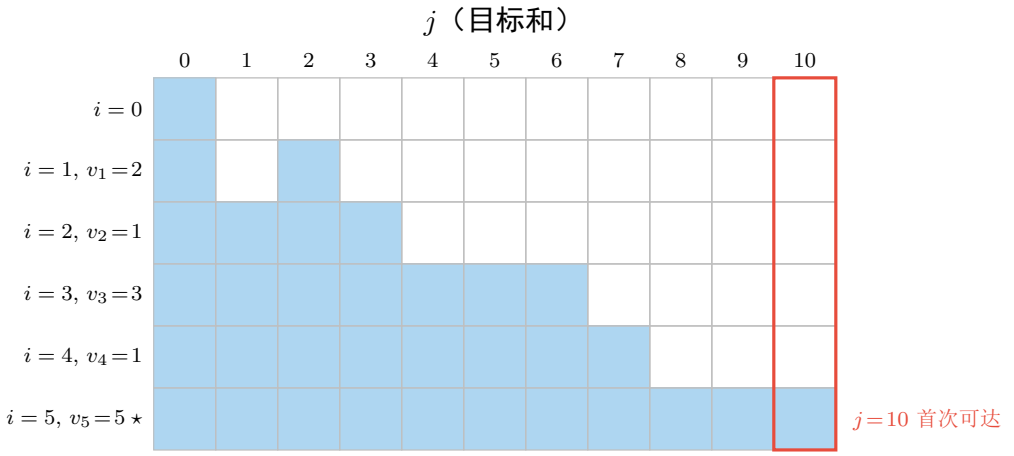


图 3: DP 状态表（$i = 0 \sim 5$，$j = 0 \sim 10$）。蓝色：dp[i][j]=true；灰色：false。红框标注目标列 $j = 10$，行 $i = 5$ 是首个使 $j = 10$ 可达的行。

3.4　代码实现（核心逻辑）

Listing 4: 动态规划求解（含回溯）

```
void dpSolve(vector<int>& v) {
    int n = v.size(), total = 0;
    for (int x : v) total += x;
    int half = total / 2;

    // 建立 (n+1) x (total+1) 的 dp 表
    vector<vector<bool>> dp(n+1, vector<bool>(total+1, false));
    dp[0][0] = true;
    for (int i = 1; i <= n; i++)
        for (int j = 0; j <= total; j++) {
            dp[i][j] = dp[i-1][j];
            if (j >= v[i-1] && dp[i-1][j-v[i-1]])
                dp[i][j] = true;
        }

    // 找最大可达和 <= half
    int best = 0;
    for (int j = half; j >= 0; j--)
        if (dp[n][j]) { best = j; break; }

    // 逆序回溯还原第一堆
    vector<int> pile1;
```

```
    int cur = best;
    for (int i = n; i >= 1; i--)
        if (!dp[i-1][cur]) {
            pile1.push_back(i);
            cur -= v[i-1];
        }
    sort(pile1.begin(), pile1.end());
    // ... 输出部分略
}
```

3.5　实验结果

功能验证　样例 $\mathbf{v} = (2, 1, 3, 1, 5, 2, 3, 4)$（$n = 8$）：两种方法均输出 $s_1 = 10$（卡牌 1、3、5）、$s_2 = 11$（卡牌 2、4、6、7、8），差值为 1，为最优解。

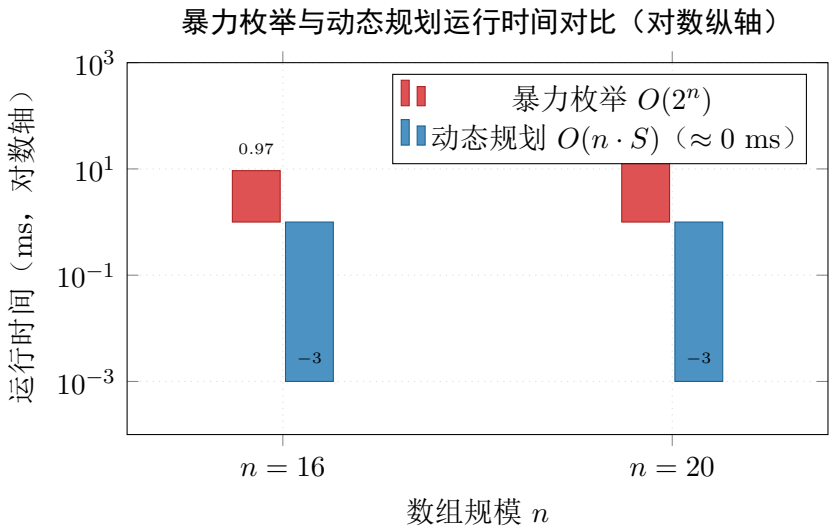


图 4: 两种算法运行时间对比。纵轴取对数；DP 运行时间低于计时分辨率（$< 1\,\mu$s），以 0.001 ms 标注。

暴力与 DP 运行时间对比

3.6　分析与讨论

图 4 直观呈现了指数级与伪多项式时间的差距：n 从 16 增至 20 时，暴力枚举耗时从 9.3 ms 增至 153.6 ms（约 $2^4 \approx 16$ 倍），完全符合 $O(2^n)$ 的增长规律；而 DP 始终在 $1\,\mu$s 以内，即便 $n = 500$（$v_i \in [1, 20]$，$S \approx 5000$）也仅需约 10 ms。

DP 的效率来自对子问题的复用：式 (1) 的转移结构使得每个” 能否凑出和 j” 的子问题只需计算一次，避免了暴力法中对相同子集的反复遍历。DP 的代价在于空间：完整二维表占用 $O(n \cdot S)$ 字节，当 S 较大时可改用一维滚动数组以换取空间效率。

4　第三题　贪心算法：供水管网最小生成树

4.1　问题建模

将 n 栋楼视为图 $G=(V,E)$ 的顶点，每条潜在管道对应一条边，权值为铺设长度。供水网络的三条要求（全连通、无环、总长最短）恰好对应图论中的**最小生成树（MST）**。

4.2　算法：Kruskal

贪心策略　按边权从小到大贪心地加入边，当且仅当该边不与已选边集形成环路。

正确性　MST 的割性质保证了此策略的全局最优性：对图的任意割（将 V 分为两个非空子集），穿越该割的最小权边必属于某棵 MST。Kruskal 每次选取当前最短的跨分量边，恰好对各割均满足此条件，因此所选边集构成 MST。

环路检测　使用带路径压缩与按秩合并的**并查集（Union-Find）**：若边 (u,v) 两端点属于同一连通分量（$\text{find}(u)=\text{find}(v)$），则加入该边会形成环，跳过；否则合并两分量。

时间复杂度　边排序 $O(m\log m)$；m 次并查集操作均摊 $O(m\,\alpha(n))$，其中 α 为反阿克曼函数，增长极缓慢，可视为常数。总体：$O(m\log m)$。

4.3　代码实现

Listing 5: 并查集 + Kruskal 主逻辑

```
struct Edge { int u, v, w; };

int par[1001], rnk[1001];
void init(int n) {
    for (int i = 1; i <= n; i++) { par[i] = i; rnk[i] = 0; }
}
int find(int x) {
    return par[x] == x ? x : (par[x] = find(par[x]));
}
bool unite(int x, int y) {
    x = find(x); y = find(y);
    if (x == y) return false;
    if (rnk[x] < rnk[y]) swap(x, y);
```

```
    par[y] = x;
    if (rnk[x] == rnk[y]) rnk[x]++;
    return true;
}

// Kruskal 主算法
sort(edges.begin(), edges.end(),
     [](const Edge& a, const Edge& b){ return a.w < b.w; });
init(n);
int total = 0, cnt = 0;
for (auto& e : edges) {
    if (unite(e.u, e.v)) {
        mst.push_back(e);
        total += e.w;
        if (++cnt == n-1) break;
    }
}
```

4.4　测试用例与结果

测试图　6 栋楼，10 条候选管道，具体数据见图 5。带权图中，红色粗线为最小生成树所选管道，灰色虚线为排除在外的候选管道。

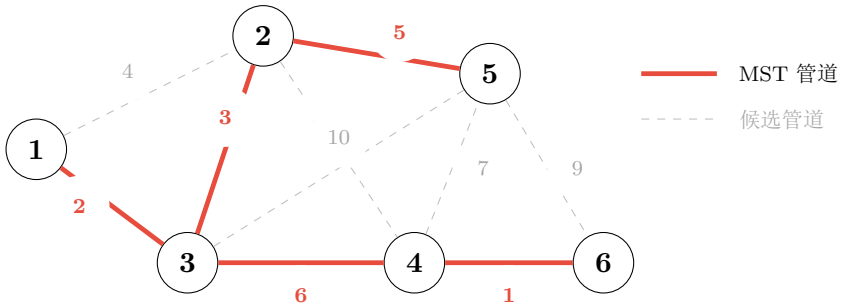


图 5: 测试用例图示（6 栋楼，10 条候选管道）。红色粗线：Kruskal 选出的最小生成树（总长 $1+2+3+5+6=\mathbf{17}$）；灰色虚线：未选边。

Kruskal 执行过程

1. 按权值排序：(4-6, 1)、(1-3, 2)、(2-3, 3)、(1-2, 4)、(2-5, 5)、(3-4, 6)、(4-5, 7)、(2-4, 8)、(5-6, 9)、(3-5, 10)。
2. (4-6, 1)：两端不连通，**加入**，分量：$\{4, 6\}$。

3. (1-3, 2)：**加入**，分量：$\{1,3\}$、$\{4,6\}$。
4. (2-3, 3)：**加入**，分量：$\{1,2,3\}$、$\{4,6\}$、$\{5\}$。
5. (1-2, 4)：1 与 2 同属 $\{1,2,3\}$，**跳过**（成环）。
6. (2-5, 5)：**加入**，分量：$\{1,2,3,5\}$、$\{4,6\}$。
7. (3-4, 6)：**加入**，所有节点并入同一分量，**完成**。

最终总长 $1+2+3+5+6=\mathbf{17}$，与程序输出一致。

4.5　分析与讨论

Kruskal 的贪心正确性依赖于割性质，该性质的成立条件是边权的可比较性与图的连通性。对于本题（同材质水管、权值为长度），条件均满足。此外，当候选管道数量 m 远小于 n^2 时（现实场景中地形限制使边稀疏），Kruskal 的 $O(m\log m)$ 比 Prim 的朴素实现 $O(n^2)$ 更具优势；若图为稠密图，则 Prim 配合 Fibonacci 堆（$O(m+n\log n)$）更优。

5　其他说明

问题 1：线性时间选择实现的一个细节　中位数算法需要在原数组中找到 pivot 值并进行划分。若数组中存在重复值，线性扫描得到的"pivot 位置" 可能不是参与分组中位数计算的那个具体元素，但由于我们只关心 pivot 的值而非其身份，选取任意一个等于 pivot 值的元素参与划分均不影响正确性。

问题 2：计时精度的干扰　在 Windows 上，`chrono::high_resolution_clock` 的实际分辨率约为 1 ms，导致部分小规模测试出现读数为零的情况。性能实验因此选取足够大的 n 以确保数据可信；对于理论上应有正值但实测为零的数据点，在图表中予以略去或注明。此外，`-O2` 优化可能将返回值未被使用的函数调用整体消除，性能测试最终使用 `volatile` 标记接收返回值以规避此问题。

问题 3：DP 的空间与回溯的取舍　DP 求解卡牌分堆时，若仅需最优差值，一维滚动数组即可；但为还原具体分配方案，需保留完整二维表以供回溯。当 n 或各 v_i 较大时，内存开销可能显著。一种折中方案是分段存储，将问题拆分为若干子区间分别处理，但实现复杂度较高，本次作业未予实现。

体会　三道题分别展示了分治、动态规划与贪心的核心设计思想。尤为值得关注的是算法理论与工程实践之间的落差：中位数算法在最坏情形上具有理论优越性，但其常数因子的劣势在随机输入下被充分放大，这说明对算法的评估不能止步于渐近符号，实际常数、缓存行为及分支预测均会对性能产生不可忽视的影响。